

Contents

1 Chapter 11 — Coalition Formation and Detection (So Long Sucker): Experiment Report	1
1.1 Part I — THE MACHINERY AND ITS EXACT ANCHORS	2
1.2 What this chapter is about	2
1.3 Experiment 1 — the SLS engine against the exact 2-player endgame	3
1.4 Experiment 2 — the coalition detector against a planted alliance	3
1.5 Experiment 3 — Shapley credit: exact toys, then SLS positions	5
1.6 Part II — LEARNING COALITIONS AND ANALYZING THE POPULATION . .	7
1.7 Experiment 4 — coalition-aware MAPPO and the alpha sweep	7
1.8 Experiment 5 — EGTA and the spinning-top: is SLS cyclic?	9
1.9 Validation harness summary	10
1.10 Prediction <-> reality reconciliation	11
1.11 Trustworthiness and sample adequacy	13
1.12 Limitations (ranked by how much they affect the conclusions)	13
1.13 Conclusions and research directions	14
1.14 Reproduction	15

1 Chapter 11 — Coalition Formation and Detection (So Long Sucker): Experiment Report

Testbed: a native, deterministic 4-player **So Long Sucker (SLS)** engine — the four-player bargaining game of Hausner, Nash, Shapley & Shubik, published in 1964 — implemented from the published rules (not cloned from any external repo), with the De Carufel & Jerade endgame analysis as the not-yet-cross-checked reference. On top of it the chapter builds a **coalition detector** (help/harm inference from chip placement), **Shapley credit** adapted to a purely-competitive game, a **coalition-aware MAPPO** self-play trainer, and an **EGTA + spinning-top** population analysis. The spinning-top (Hodge) decomposition is reused wholesale from Chapter 10 and the projected-meta-game meta-Nash from Chapter 9; the coalition detector re-derives Chapter 7’s opponent-modeling principle from scratch. The one **exact** anchor is a 2-player **minimax endgame** solver used to certify the engine.

PhD connection: this is the thesis *frontier*. Three hooks: the **coalition detector** lifts opponent modeling from “what kind of player is this?” to “who is allied with whom?” (Contribution #1); the **N-player safe baseline** loses its Nash anchor — with no minimax value and an empty core there is no stable allocation for safety to rest on; a behavioral anchor in the spirit of piKL is a candidate, not a guarantee (Contribution #2, the gap this chapter *frames* but does not close); and the **EGTA meta-game + Shapley credit** stand in for exploitability, which with more than two players no longer measures what an agent is guaranteed, least of all against a coalition (Contribution #3).

Scope of results: every number in this report is **measured from a real run** and read from the artifacts under `implementation/results/` — `smoke_results.json`, `sweep_smoke.json`,

`sweep_scale.json` — and the measured-vs-predicted dev log `EXECUTION_NOTES.md` (both relative to `implementation/step11/`). The cooperative-game-theory toys are bracketed by their *exact* textbook values (glove, 3-player majority); the SLS environment is bracketed by the *exact* 2-player minimax endgame. Per WORKFLOW §0.1, contradicted predictions are **kept as stated and reconciled** with what actually happened (*Prediction <-> reality reconciliation*) — including a real engine bug found and fixed mid-run.

Artifact caveat (read once). `results/scale_results.json` is a **pre-fix** tournament run (symmetric spread 0.525, hero win ~0.83, `shapley_higher_coalition:false`, `alpha=0.3`). It is cited below **only** as evidence of the seat-0 bug (the *tie-break bug* reconciliation). Its post-fix successor was not regenerated, so the authoritative scale-tier numbers come from the 5-seed `sweep_scale.json`, and the authoritative single-config numbers from the post-fix `smoke_results.json`.

How to read this report. Both parts follow the same arc: **what we test -> how -> results -> conclusion**. **Part I** builds and *certifies the machinery*: the SLS engine against the exact endgame, the coalition detector against a planted alliance, and Shapley credit against exact cooperative-game toys and hand-set SLS positions. **Part II** *learns and analyzes coalitions*: coalition-aware MAPPO and the 5-seed `alpha` sweep, then the EGTA / spinning-top population structure. *Prediction <-> reality reconciliation* reconciles the contradicted predictions (the tie-break bug, the `alpha` dead zone, the cyclic-but-not-dominant structure, the coalition-vs-winning trade-off); *Trustworthiness*, *Limitations* and *Conclusions* cover trust, limitations, and directions; *Reproduction* lists reproduction commands.

1.1 Part I — THE MACHINERY AND ITS EXACT ANCHORS

1.2 What this chapter is about

Chapters 2-10 all leaned on one thing: a **2-player** game with an **exact** best-response / exploitability oracle, so “did it work?” was a single number. Chapter 11 removes that. The moment a third player joins, **alliances** become possible — form them, exploit them, betray them — and Nash equilibrium becomes both *intractable* (N=4 FFA) and *without guarantees* (it guards only against unilateral deviations, not against coalitions). So the chapter trades exact evaluation for **empirical** evaluation (win-rate, coalition score, EGTA cyclic ratio) and keeps exactly one exact anchor: the analytically solvable **2-player endgame**. Everything in Part I exists to make the later learning claims trustworthy — certify the environment, certify the detector, certify the credit assignment — before asking a population to learn in it.

1.3 Experiment 1 — the SLS engine against the exact 2-player endgame

What we test. Does the native SLS engine (a) always terminate, (b) stay zero-sum, and (c) agree with an **exact minimax** solver on the 2-player endgame — the one subgame with a ground-truth winner? (Raw step validation, coalition-env correctness.)

How. Play 100 random 4-player games and check termination + zero-sum rewards. Separately, enumerate small 2-player endgame positions and compare the engine’s optimal-play winner to a full minimax search. Data: `results/smoke_results.json` (env block).

Check	Measured	Verdict
Random games terminate	100/100 terminated	pass
Rewards zero-sum	all zero-sum	pass
Engine vs minimax endgame	<code>endgame_mismatches: 0</code>	pass

Results. Zero endgame mismatches, all games terminate, all rewards sum to zero. The engine is a faithful, self-consistent SLS implementation *for its own ruleset*. One honesty caveat travels forward: the minimax check certifies the engine against **its own** documented rules (`capturer-plays-next`, `empty-hand skip`, `deadlock most-chips tie-break`), not against a line-by-line transcription of De Carufel & Jerade — whose theorem statements remain a verify-when-you-read-it item (*Trustworthiness and sample adequacy*). This distinction turns out to matter (the *tie-break bug* reconciliation).

Conclusion. The environment is certified where an exact answer exists. Because there is no exact oracle for N=4, this 2-player anchor is the only ground truth the rest of the chapter can lean on — which is exactly why the engine’s *simplifications*, not its solver, are the prime suspects when a symmetric setup later behaves asymmetrically.

1.4 Experiment 2 — the coalition detector against a planted alliance

What we test. Can the detector *recover a coalition it was never told about*, purely from the moves? (Raw step validation; Contribution #1.)

How. Two players are scripted to systematically help each other (place chips into each other’s piles) and harm the others (capture). The detector builds `help` and `harm` matrices from the `MoveEvent` log, forms `net_support`, and reports the strongest pair. Data: `results/smoke_results.json` (detector block).

Quantity	Measured	Verdict
Planted allies	<code>{0,1}</code>	—

Quantity	Measured	Verdict
Strongest detected pair	$\{0, 1\}$	correct
Pair coalition score	10.0	clean separation
Coalition matrix	$[0, 1]=10.0$; all cross-pair entries 0 or -1	clean

Results. The detector recovers the planted $\{0, 1\}$ coalition exactly, with a strong positive score (10.0) on the allied pair and zero/negative net support everywhere else. The alliance is legible entirely from the chip-placement stream — in this engine the alliance is *encoded entirely in the moves* (placing a chip is a handshake, capturing a pile is the knife): the free, unenforceable negotiation of the real game is not modelled. This is a sanity check against one scripted, overt pair; false positives on non-colluding play and covert colluders were not tested.

Conclusion. Opponent modeling generalizes cleanly from “infer the opponent’s type/hand” (Chapter 7) to “infer the social structure” (who is allied with whom). This is Contribution #1 made concrete on a real game, and it is the input signal the coalition-aware trainer (*Experiment 4*) rewards.

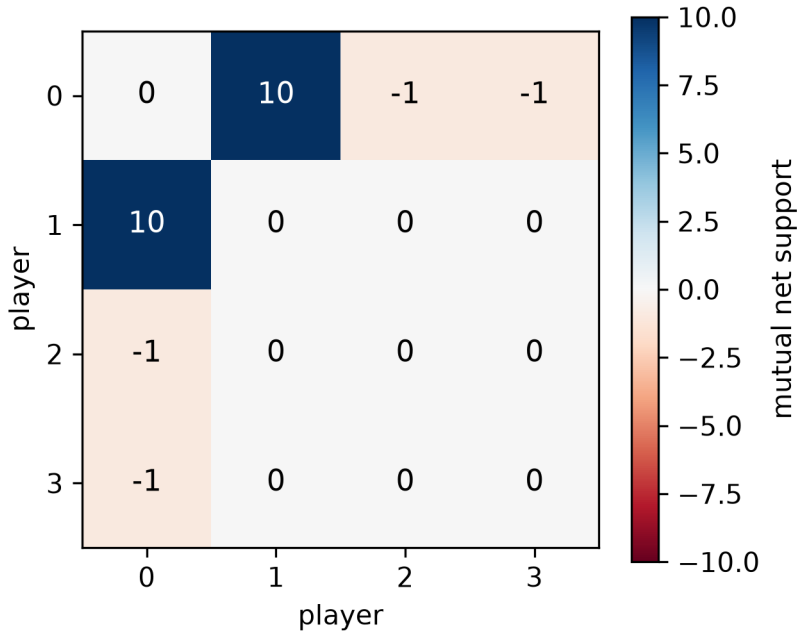


Figure 1: Coalition-score matrix of the planted-alliance test (`smoke_results.json`), inferred purely from chip placement: the planted alliance between players 0 and 1 shows as strong mutual support (10), while the other pairs are neutral (0) or hostile (-1). The detector reads the alliance straight off the move stream; in this engine negotiation is not modelled.

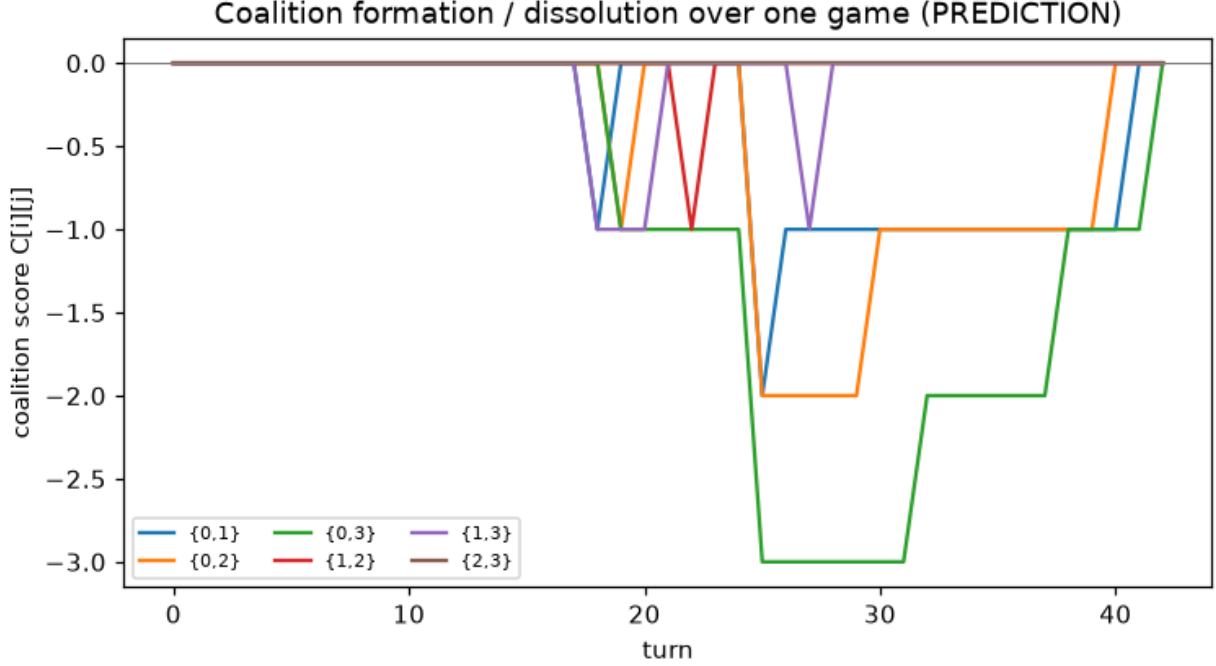


Figure 2: Pairwise coalition score over one sample game of the heuristic baseline pool (an older plot; its title still reads PREDICTION): only hostile (negative) scores appear, and they return to 0 by the end of the game - no alliance forms in this pool. A temporal view like this is what a learning agent would condition on to time an exploit-then-break.

1.5 Experiment 3 — Shapley credit: exact toys, then SLS positions

What we test. (a) Does the Shapley implementation reproduce the *exact* textbook values on cooperative-game toys (glove, 3-player majority), including the **core** (stability) test? (b) Adapted to SLS — where “coalition value” is redefined as *the probability a coalition member wins* — does a **symmetric** position give **equal** credit, and does an **asymmetric** strong-pair position concentrate credit on the strong pair? (Raw step validation; Contribution #3.)

How. Exact Shapley by enumeration on the toys; core emptiness by LP feasibility. On SLS, credit is the Shapley value of the win-probability characteristic function, estimated by Monte-Carlo rollouts (shapley_rollouts=300). Data: results/smoke_results.json (shapley block).

Case	Prediction	Measured	Verdict
Glove game Shapley	(2/3, 1/6, 1/6)	[0.6667, 0.1667, 0.1667]	exact
Glove core	non-empty, allocation (1, 0, 0)	non-empty, alloc [1, 0, 0]	exact

Case	Prediction	Measured	Verdict
3-player majority Shapley	$(1/3, 1/3, 1/3)$	$[0.3333, 0.3333, 0.3333]$	exact
3-player majority core	empty	core non-empty = False	exact
SLS symmetric credit	spread < 0.15	$[0.247, 0.257, 0.253, 0.243]$ spread 0.013	pass (after the <i>tie-break bug</i> reconciliation fix)
SLS asymmetric [8, 8, 1, 1]	strong pair dominates	strong-pair credit 1.0 vs weak 0.0; $v(\{0, 1\}) = 1.0$	pass

Results. The cooperative-GT toys are reproduced to the fourth decimal, *including* the two structural facts that matter for the thesis: the glove game’s core is **non-empty** (a stable split exists), while the 3-player majority game’s core is **empty** (no allocation is stable — any pair can profitably defect). On SLS, a genuinely symmetric position gives near-equal credit (spread 0.013), and an asymmetric strong-pair position hands *all* credit to the strong pair — the coalition value of $\{0, 1\}$ is 1.0. The symmetric result is reported **after** a real bug fix (the *tie-break bug* reconciliation): on the first run the spread was 0.54, a red FAIL that turned out to be an engine tie-break artifact, not a Shapley error.

Conclusion. The Shapley machinery is trustworthy in its own right — it passes the exact fairness *and* stability tests — and, adapted to a competitive game via win-probability, it behaves correctly on symmetric and asymmetric SLS positions. The **empty core** of the majority game is the SLS situation in miniature: SLS is an essential constant-sum game, so its core is empty too — *no stable allocation exists, so coalitions will break* — and together with the absence of a minimax value beyond two players this is why N-player “safe” play cannot be anchored to a stable equilibrium (Contribution #2).

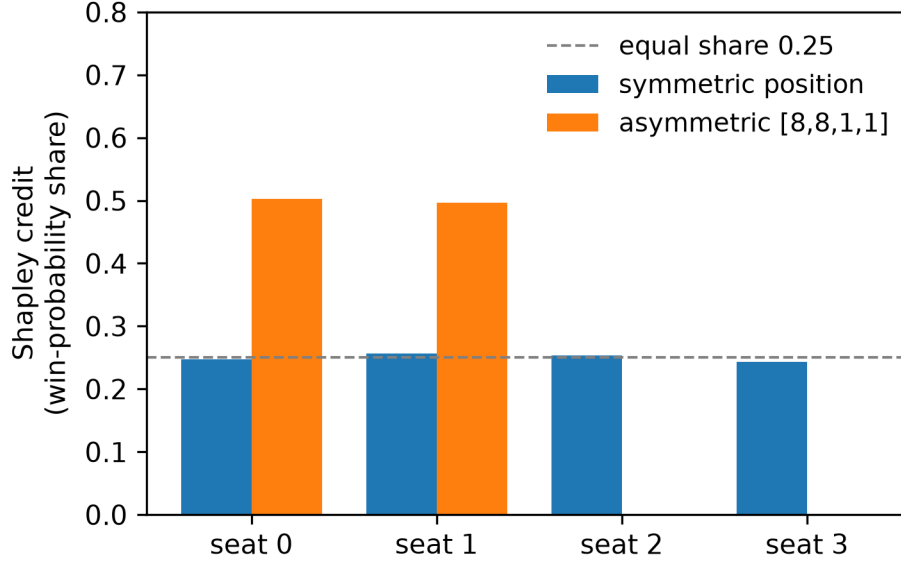


Figure 3: Shapley credit on two SLS positions (`smoke_results.json`): per-seat Shapley value of the win-probability characteristic function. In the symmetric position credit is near-equal across the four seats (spread 0.013 after the fix); in the asymmetric position `[8, 8, 1, 1]` the strong pair absorbs all credit ($0.503 + 0.497$) and the weak pair gets 0.0. Credit follows the real advantage only once the engine’s tie-break rule is unbiased.

1.6 Part II — LEARNING COALITIONS AND ANALYZING THE POPULATION

1.7 Experiment 4 — coalition-aware MAPPO and the alpha sweep

What we test. Does blending the sparse winner reward with **Shapley-decomposed coalition credit** produce agents that *form coalitions more* than sparse-reward agents — and, critically, *under what blend weight?* (Raw step validation L560: coalition-forming is the primary target, winning secondary.)

How. A masked, episodic PPO seat is trained by self-play with reward $r = \alpha \cdot r_{\text{sparse}} + (1 - \alpha) \cdot \text{credit}$ ($\alpha = 1$: win reward only; $\alpha = 0$: credit only), where **credit** is either the cheap critic-value proxy or the expensive rollout **counterfactual** Shapley. The **coalition score** is the trained population’s mean detector score. The single-config runs use the default `alpha=0.3`; a **5-seed paired** grid (`sweep.py`) then sweeps `alpha` x `credit_mode` x `synergy` at two tiers, reporting the paired gap `gap = coalition_score(shapley) - coalition_score(sparse)` with error bars. Data: `results/smoke_results.json` (training), `results/sweep_{smoke,scale}.json`.

Single-config (default $\alpha=0.3$):

Method	Coalition score (smoke)	Win rate vs random (smoke)
Random baseline	0.000	0.250
MAPPO (sparse)	0.00125	0.44
MAPPO + Shapley (this chapter)	0.00417	0.41

5-seed paired sweep (** = mean gap > 2·SE, which with 5 seeds is roughly $p < 0.12$; sparse baseline coalition score: smoke 0.0073, scale 0.0109):

Tier	Credit	α	Synergy	Gap (shapley - sparse)	Sig
scale (7/1500)	proxy	0.0	0.3	+0.0376 ± 0.0103	** (score ~4.5x sparse)
scale	proxy	0.0	0.1	+0.0305 ± 0.0130	**
scale	counterfactual	0.0	—	+0.0128 ± 0.0026	**
scale	any	≥ 0.3	*	−0.0003 ... − 0.0035	— (all negative)
smoke (5/400)	proxy	0.0	0.1	+0.0024 ± 0.0008	** (tiny)
smoke	any	≥ 0.3	*	−0.0031 ... + 0.0012	— (none significant)

Results. At the default $\alpha=0.3$, the Shapley agent beats sparse at smoke (0.00417 > 0.00125) but only marginally, and — on the pre-fix single scale run — the direction reversed (the *alpha dead zone* reconciliation). The sweep resolves the ambiguity: **α is the dominant knob, and 0.3 sits in a dead zone.** The coalition score rises *significantly* only at low α : at $\alpha \approx 0$, proxy credit with synergy=0.3 gives a paired gap of +0.0376 ± 0.0103 (the Shapley agents’ score 0.0484 is ~4.5x the sparse baseline 0.0109), while **every scale-tier $\alpha \geq 0.3$ cell is negative.** The effect is *larger* in the scale tier (12x the smoke gap at synergy=0.1; game size and training length change together, so the two are confounded), and the **cheap critic-value proxy beats the expensive counterfactual** (+0.038 vs +0.013 at scale).

Conclusion. The low- α effect is **real and robust in the sweep, but it is not yet evidence of learned coalitions.** As implemented, the credit carries no coalition information: the rollout value $v(S) = \sum_{i \in S} P(i \text{ wins})$ is additive, so each player’s Shapley credit is simply its own win

probability (computed once per batch, from the initial state), and the centered proxy is each agent’s own critic value minus the table mean; the coalition score (mean $|C_{ij}|$) also rises with mutual hostility. A coalition-free dense-reward control is needed before the effect can be read as coalition formation. The fix is “weight the coalition credit heavily” ($\alpha \approx 0.05-0.1$), not “compute a truer credit.” And it costs something: pure coalition credit ($\alpha=0$) drops win-rate to ~ 0.29 (near the 0.25 random floor), while $\alpha \geq 0.1$ keeps it ~ 0.52 (the *coalition-vs-winning* reconciliation) — coalition-forming is bought with competitive performance, exactly as the raw step framed it.

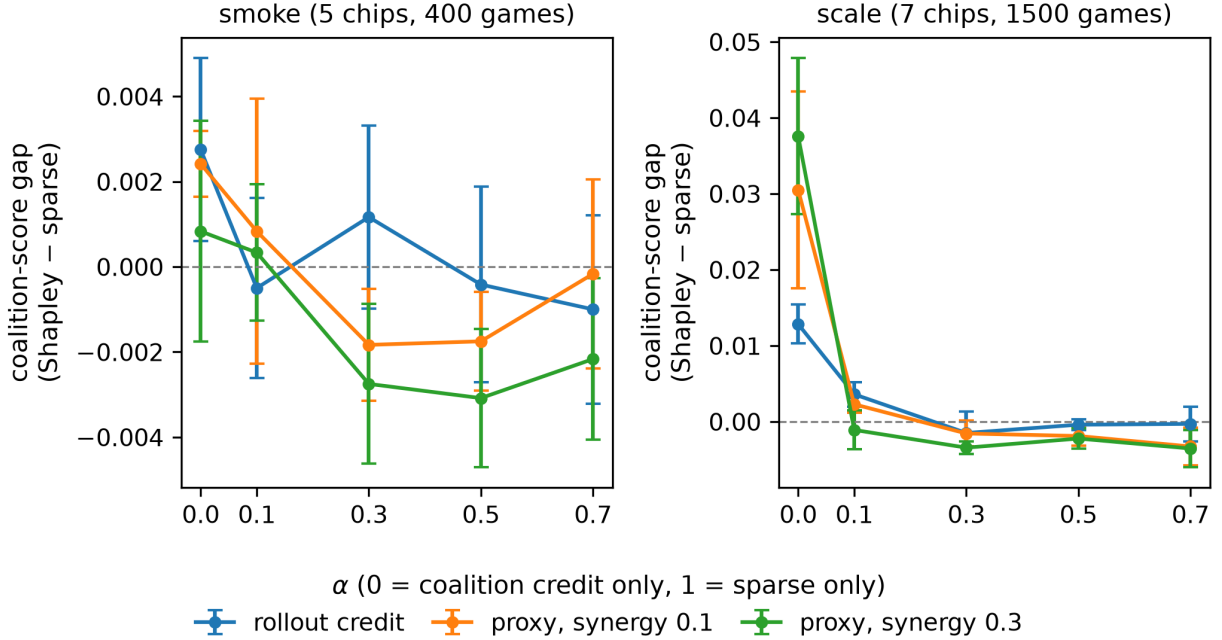


Figure 4: Paired coalition-score gap (Shapley minus sparse) across the alpha x credit x synergy sweep, 5 seeds with error bars. The gap is large and significant only in the low-alpha regime (peaking at +0.038 with the proxy credit at $\alpha=0$; the Shapley agents’ score is $\sim 4.5\times$ the sparse baseline’s) and, in the scale tier, goes negative for every $\alpha \geq 0.3$. The earlier null result came from measuring at $\alpha=0.3$ - the dead zone.

1.8 Experiment 5 — EGTA and the spinning-top: is SLS cyclic?

What we test. Treating whole SLS strategies as the atoms of a meta-game, is the resulting population **cyclic** (a wheel of coalition counters, as Chapter 10 predicted for FFA games) or **transitive** (a skill ladder)? (Raw step validation L561.)

How. Play every pair of strategies to fill an empirical payoff matrix, project the 4-player payoff **tensor** to a 2-player pairwise matchup matrix (so Chapter 09’s `solve_meta_nash` and Chapter 10’s

Hodge `spinning_top` still apply), and report the transitive/cyclic ratio. Two pools are decomposed: the default **skill-ladder** baseline pool, and a **coalition pool** (`fixed_ally_1/2/3` + `betray_1` + `random`). Data: `results/smoke_results.json` (egta), `EXECUTION_NOTES.md` (check 5).

Population	Transitive	Cyclic	Structure
Skill-ladder pool (smoke)	0.9675	0.2529	transitive-dominant
Skill-ladder pool (scale)	0.951	0.308	transitive-dominant
Coalition pool (60 games/cell)	–	≈ 0.57	strong cyclic component
Coalition pool (200 games/cell)	–	≈ 0.69	strong cyclic component

Results. As in Chapter 10, *which population you decompose decides what you see*. The default skill-ladder pool is transitive-dominant (cyclic 0.25-0.31), but a coalition pool pushes the cyclic ratio to **~ 0.57 - 0.69** — a large non-transitive component that strongly confirms the *direction* of raw L561. It stays **honestly red** under the strict “>50% dominance” threshold (cyclic² still just under 0.5; transitive marginally larger, and the pool was not tuned to cross the line).

Conclusion. Coalition strategies make SLS *meaningfully non-transitive and near-balanced*, but not strictly *cyclic-dominant* at this scale. The likely residual is that the **2-type pairwise projection discards 3-/4-player coalition effects** — a tensor-native or 3-type decomposition is the open modeling question (the *cyclic-but-not-dominant* reconciliation, *Conclusions and research directions*). The Chapter 10 caveat repeats verbatim: choosing how you build the population is choosing whether you see a wheel or a ladder.

1.9 Validation harness summary

The `validate.py` harness encodes the raw step’s PASS/FAIL targets. After the the *tie-break bug* reconciliation engine fix:

#	Check	Result	Verdict
1	SLS env (endgame vs minimax / termination / zero-sum)	0 mismatches, terminated, zero-sum	PASS

#	Check	Result	Verdict
2	Coalition detection (planted {0,1})	strongest pair {0,1}, score 10.0	PASS
3	Shapley (toys + symmetric spread + asymmetric)	glove/majority exact; spread 0.013; strong-pair dominates	PASS
4	Coalition-aware training (smoke)	shapley 0.004 > sparse 0.001	PASS
5	Spinning-top (strict cyclic > 50% dominance)	coalition pool cyclic ~0.57-0.69, below strict threshold	FAIL (honest)

Net: validate.py --config smoke = 4/5 PASS. Check 5 is left honestly red under the strict dominance rule (Chapter 10 likewise shipped an honest red FAIL). The machinery (env, detector, Shapley) is fully certified; the single red is a *game-structure* finding, not a code defect.

1.10 Prediction <-> reality reconciliation

Per WORKFLOW §0.1, contradicted predictions are kept and reconciled, not silently edited. Four gaps — one of which was a genuine bug caught by suspecting the engine before the prediction.

1.10.1 The two red FAILs shared one root cause: a deadlock tie-break bug, not first-mover order

Predicted: a symmetric position gives symmetric outcomes (check 3 spread <0.15; check 5 sees the cyclic coalition structure). *Measured (first run):* check 3 spread 0.54 and check 5 transitive 0.998 — both red — with a monotone **seat-0 advantage** across three independent scripts (all-random winners [94,42,33,31]; symmetric Shapley credit [0.593,0.24,0.113,0.053]; the same fixed-ally strategy scoring 0.493 in seat 0 vs 0.243 in seat 1). *Reconciliation:* suspecting the engine before the prediction (§0.1), diagnostics found the mechanism: **~99.5% of random games in this engine end in a deadlock** (all live hands empty at ~28 turns — a consequence of its simplified rules, which skip a player with an empty hand), so the winner is decided by **_most_chips** — whose **lowest-index tie-break** handed seat 0 ~2x its fair share (rotating the start seat changed nothing; the bias was the tie-break, not turn order). Fix: an **unbiased random deadlock tie-break** threaded through `sls_game.apply(..., rng=...)` on the play/eval/train paths, with the exact endgame minimax kept deterministic (and `verify_endgame_consistency` switched to a determinis-

tic optimal-vs-optimal rollout so it still matches the minimax tree). Result: symmetric spread **0.54** → **0.013** (check 3 FAIL→PASS), all-random winners now uniform ($[0.251, 0.238, 0.251, 0.260]$). A side effect exposed a second artifact: the impressive ~ 0.87 hero win-rate was *itself* the seat-0 tie-break (the hero always sat in seat 0); the fair number is ~ 0.41 vs the 0.25 floor. Lesson: in a game that almost always ends in a near-tie, the **tie-break rule is load-bearing**, and a symmetric *position* is not a symmetric *outcome* until it is unbiased.

1.10.2 “Coalitions don’t emerge at scale” was overturned — it was a mis-set blend weight

Predicted: Shapley credit yields more coalition behavior than sparse reward. *Measured (single-config):* true at smoke ($0.0042 > 0.0013$) but **collapsing/reversing at scale** on the pre-fix run — read at the time as “the proxy credit is too weak once training is longer.” *Reconciliation:* the 5-seed paired sweep refutes that. **alpha** is the dominant knob and **0.3 sits in the dead zone**; the coalition score rises significantly **only at low alpha** (at $\alpha \approx 0$, proxy/synergy=0.3 gives $+0.0376 \pm 0.0103$; score $\sim 4.5\times$ sparse), while **every scale-tier $\alpha \geq 0.3$ cell is negative** (the sparse winner-takes-all term suppresses the coalition signal). Two further surprises: the effect is **larger in the scale tier** (game size and training length confounded; opposite to the naive “smoke-positive / scale-null” read, which was an artifact of holding $\alpha=0.3$ at both tiers), and the **cheap critic-value proxy beats the expensive counterfactual** ($+0.038$ vs $+0.013$). So the effect is **real and robust in the right regime** — though, as implemented, the credit carries no coalition information (Experiment 4, conclusion) — and the fix is “weight the coalition credit heavily,” not “compute a truer credit.” Methodologically this echoes Chapters 9-10: **a single config hides what a seeded sweep reveals**.

1.10.3 The SLS meta-game is strongly cyclic, but not (yet) strictly cyclic-dominant

Predicted (from Chapter 10): FFA coalition games have a large cyclic component. *Measured:* it depends entirely on **which population** you decompose. The default **skill-ladder** pool is transitive-dominant (cyclic $0.25/0.31$), but a **coalition pool** pushes cyclic to $\sim 0.57\text{--}0.69$ — a large non-transitive component that strongly confirms the *direction* of raw L561, while staying **honestly red** under the strict “>50% dominance” threshold (cyclic² just under 0.5). *Reconciliation:* not a bug — the same Chapter 10 lesson that structure is a property of the *population*. The likely residual is that the **2-type pairwise projection discards 3-/4-player coalition effects** (raw L600) — a consolidation-level modeling question, not a solver error.

1.10.4 Coalition behavior is bought with competitive performance (a genuine trade-off)

Predicted (raw L560): coalition-forming is the primary target, winning secondary. *Measured:* pure coalition credit ($\alpha=0$) drops hero win-rate to ~ 0.29 (near the 0.25 random floor), while $\alpha \geq 0.1$ keeps it ~ 0.52 . *Reconciliation:* the prediction holds and is now quantitative — you do not get coalition behavior for free; the α knob is exactly the dial that trades competitive performance for social behavior.

1.11 Trustworthiness and sample adequacy

- **The exact anchors are real.** The cooperative-GT toys (glove, majority) reproduce their textbook Shapley values *and* core (stability) results to four decimals; the SLS engine has **zero** mismatches against the exact 2-player minimax endgame. These are deterministic and exactly reproducible.
 - **The detector result is a clean recovery**, not an estimate: a planted $\{0,1\}$ coalition is recovered exactly with a well-separated score (10.0 vs 0/-1).
 - **The training claims rest on a 5-seed paired sweep**, not a single run. The significant cells (**) have mean gaps exceeding twice their standard error; the *directions* (low- α positive, scale-tier $\alpha \geq 0.3$ negative, proxy $\geq$ counterfactual, effect larger in the scale tier) are the trustworthy claims, not third-decimal magnitudes. With 5 seeds, a mean above 2 SE corresponds to roughly $p < 0.12$; the headline cells clear 3 SE.
 - **Engine fidelity is the standing caveat.** The engine is certified against *its own* rules, not a transcription of De Carufel & Jerade; the seat-0 bug (the *tie-break bug* reconciliation) proves the tie-break rule genuinely changes outcomes, so this reconciliation is not cosmetic.
 - **The stale scale_results.json** is a pre-fix artifact used only to demonstrate the *tie-break bug* reconciliation; it is not a source for any scale-tier claim.
-

1.12 Limitations (ranked by how much they affect the conclusions)

1. **The credit carries no coalition information (Experiment 4)** — the rollout credit is each player’s own win probability (an additive game), the proxy each agent’s own critic value minus the table mean, and the coalition score counts hostile as well as supportive interactions. The low- α effect shows that a dense reward changes interaction patterns, not that coalitions are learned; a credit built on the detector’s help/harm matrix and a coalition-free dense-reward control are needed.
2. **Engine rule fidelity vs the source paper (the *tie-break bug* reconciliation,**

Trustworthiness and sample adequacy) — the seat-0 tie-break bug and the still-red cyclic check both trace to the engine’s documented simplifications (`_most_chips` tie-break, `capturer-plays-next`, `empty-hand skip`). Reconciling the turn/tie-break model against De Carufel & Jerade is the highest-value follow-up; it is where correctness *and* the cyclic signal both improve.

3. **The 2-type pairwise projection (the *cyclic-but-not-dominant* reconciliation)** — collapsing a 4-player payoff tensor to head-to-head pairs likely discards the 3-/4-player coalition cycling, which is the prime suspect for the cyclic ratio sitting just under strict dominance.
4. **α is a design choice, not tuned in the shipped default (the *alpha dead zone* reconciliation)** — the code keeps $\alpha=0.3$ (the dead zone) with its honest marginal result; the evidence-based $\alpha\approx 0.05-0.1$ recommendation is left for a human decision (§0.1: report, don’t rig).
5. **Proxy-credit generality (the *alpha dead zone* reconciliation)** — the cheap proxy beats the counterfactual *on SLS at low α* ; whether that holds on other games is untested.
6. **Toy scale throughout** — 4-7 chips, small nets; nothing here should be extrapolated to large N-player games.

1.13 Conclusions and research directions

Conclusions. Part I certifies the machinery: the SLS engine matches the exact 2-player minimax endgame (0 mismatches), the coalition detector recovers a planted alliance exactly ($\{0,1\}$, score 10.0), and Shapley credit reproduces the exact cooperative-GT toys — *including the empty core* of the majority game that is the structural signature of unstable coalitions. Part II learns and analyzes coalitions: coalition-aware MAPPO raises the coalition score over sparse reward — robustly (+0.038, 5 seeds) but only at **low α** , and at a measured competitive cost; as implemented, though, the credit carries no coalition information, so this is not yet evidence of learned coalitions. The SLS meta-game has a **strong cyclic component** for a coalition pool ($\sim 0.57-0.69$), though the transitive part stays slightly larger. Two predictions broke honestly and one bug was caught: the “symmetric” game hid a tie-break artifact, and “coalitions don’t emerge at scale” was a mis-set blend weight.

Research directions (each tied to a measured effect):

- *Reconcile the engine turn/tie-break model against De Carufel & Jerade*, then re-check spinning-top dominance — the *tie-break bug* reconciliation and the *cyclic-but-not-dominant* reconciliation likely share this root.
- *Adopt $\alpha\approx 0.05-0.1$ as the coalition-training default* — evidence in the sweep (the *alpha dead zone* reconciliation), left as a human design decision.
- *Try a 3-/4-player-aware EGTA projection* (not the 2-type collapse) to see whether coalition cycling crosses the strict >50% line (the *cyclic-but-not-dominant* reconciliation).

- *Formalize N-player safety without a Nash/core anchor (Contribution #2)* — the empty core (*Experiment 3*) removes core-based stability, and piKL gives a behavioral baseline with no exploitability bound; this is the thesis frontier the chapter frames but does not close.
 - *Build a coalition-carrying credit from the detector’s help/harm matrix and test it against a coalition-free dense-reward control* — the real fix for the Experiment 4 caveat, and C2 research for the thesis.
 - *Regenerate the stale post-fix `results/scale_results.json` so the committed scale tournament matches the sweep.*
-

1.14 Reproduction

From `implementation/step11/implementation/` with the project `.venv` active:

```
# validation harness (PASS/FAIL against the raw-step targets)
```

```
python validate.py --config smoke
```

```
# single-config tournament -> results/{smoke,scale}_results.json
```

```
python tournament.py --config smoke
```

```
python tournament.py --config scale
```

```
# the 5-seed paired alpha sweep -> results/sweep_{smoke,scale}.json + plots/sweep_coalition_gap
```

```
python sweep.py --tier smoke
```

```
python sweep.py --tier scale
```

```
# chapter figures from the saved results only (no games, no training)
```

```
python plotting.py
```

Seeds are fixed in `config.py` and in `sweep.py`. The env, detector, and cooperative-GT toy results are exactly reproducible; the training results are direction-stable across the 5 seeds. Every number in this report was read from `results/{smoke_results.json, sweep_smoke.json, sweep_scale.json}` and the measured-vs-predicted dev log `EXECUTION_NOTES.md`; the pre-fix `scale_results.json` is cited only as evidence of the seat-0 tie-break bug (the *tie-break bug* reconciliation).